

SC4052 Cloud Computing Exam Notes (High-Yield)

This note is a compact closed-book revision sheet built from past paper patterns (2023-2025) and lecture summaries.

0) Exam Pattern: What Comes Out Repeatedly

1. Virtualization comparison (full vs para vs hardware-assisted).
 2. Clos/fat-tree topology construction + blocking condition.
 3. RAID / erasure coding reliability-capacity tradeoff.
 4. CPU scheduling math: EDF/SEDF schedulability, credit scheduler shares.
 5. Network/TCP AIMD throughput derivation.
 6. Security math: Diffie-Hellman, RSA signature/verification.
 7. CAP theorem applied to real systems.
 8. PageRank (with/without teleportation).
 9. Consistent hashing event tracing and rebalancing.
-

1) Virtualization (Fast Compare)

Key terms

- Full virtualization: trap-and-emulate / binary translation, unmodified guest OS.
- Para-virtualization: guest OS modified, hypercalls, better performance.
- Hardware-assisted: Intel VT-x/AMD-V, unmodified guest OS, best practical cloud performance.

Typical exam compare points

- Compatibility: Full/HW-assisted high, Para lower (needs guest changes).
 - Performance: HW-assisted > Para > Full (legacy full virtualization).
 - Operational fit: Hosted for desktop/dev, bare-metal hypervisor for cloud production.
-

2) Clos and Fat-Tree

2.1 3-stage Clos setup

Given N, n, k, m :

- Number of first-stage switches: $r = N/n$.
- Stage 1 size: $n \times k$.
- Stage 2 count: k switches of size $m \times m$.
- Stage 3 count: r switches of size $k \times n$.

Blocking conditions (must memorize)

- Strict-sense nonblocking: $m \geq 2n - 1$.
- Rearrangeably nonblocking: $m \geq n$.

So if $n = 4, m = 4$: rearrangeably nonblocking yes, strict-sense nonblocking no.

2.2 Fat-tree formulas (k-port)

- Pods: k .
- Core switches: $(k/2)^2$.
- Hosts per edge switch: $k/2$.
- Total hosts: $k^3/4$.
- Total switches (common approximation): $5k^2/4$.

Exam trick: if your answer does not satisfy $\text{hosts} = k^3/4$, your wiring/count is likely wrong.

3) RAID and Erasure Coding

3.1 Reliability intuition

- RAID 0: no redundancy, loss if any disk fails.
- RAID 5: survives 1 disk failure, loss when 2+ fail.
- RAID 6: survives 2 disk failures, loss when 3+ fail.
- Replication (2x, 3x): better resilience but larger storage overhead.

3.2 Quick formulas (small p)

For n disks, independent failure probability p :

- RAID 0: $P_{loss} = 1 - (1 - p)^n \approx np$.
- RAID 5: $P_{loss} \approx \binom{n}{2} p^2$.
- RAID 6: $P_{loss} \approx \binom{n}{3} p^3$.

3.3 Capacity efficiency

- RAID 5 usable fraction: $(n - 1)/n$.
- RAID 6 usable fraction: $(n - 2)/n$.
- Replication r -copy usable fraction: $1/r$.

Tradeoff statement template:

- More redundancy -> lower failure probability.
 - More redundancy -> lower unique usable capacity.
-

4) CPU Scheduling (EDF / SEDF / Credit)

4.1 Schedulability test (single CPU)

For periodic/reservation tasks (s_i, p_i) :

$$U = \sum_i \frac{s_i}{p_i}$$

Schedulable (necessary and sufficient for EDF with implicit deadlines):

$$U \leq 1$$

4.2 Hyperperiod

$$H = \text{lcm}(p_1, p_2, \dots, p_n)$$

Useful for counting CPU quanta per VM in one full cycle.

4.3 SEDF tuple

(s_i, p_i, x_i) :

- $x_i = 0$: non-work-conserving.
- $x_i = 1$: work-conserving (can consume slack).

4.4 Credit scheduler share

If weights are w_i :

$$share_i = \frac{w_i}{\sum_j w_j}$$

If total credits in a window is C :

$$C_i = C \cdot \frac{w_i}{\sum_j w_j}$$

Fast ratio method: weights 256:512:256 -> shares 1:2:1.

5) TCP AIMD Throughput (Very Common Derivation)

Assume sawtooth with one loss per cycle and no timeout.

- Window at loss: W .
- Minimum after MD: $W/2$.
- Sawtooth time:

$$STT = \frac{W}{2} RTT$$

Average window:

$$\bar{W} = \frac{W/2 + W}{2} = \frac{3W}{4}$$

Packets per cycle:

$$\frac{3W^2}{8}$$

Loss rate:

$$p = \frac{8}{3W^2} \Rightarrow W = \sqrt{\frac{8}{3p}}$$

Throughput:

$$T \approx \frac{\bar{W} \cdot MTU}{RTT} = \sqrt{\frac{3}{2}} \frac{MTU}{RTT \sqrt{p}} \approx 1.22 \frac{MTU}{RTT \sqrt{p}}$$

Interpretation: throughput scales as $1/\sqrt{p}$.

6) Security Math: Diffie-Hellman + RSA

6.1 Diffie-Hellman

Public: prime p , generator g .

- Alice secret a , sends $A = g^a \bmod p$.
- Bob secret b , sends $B = g^b \bmod p$.
- Shared secret:

$$s = B^a \bmod p = A^b \bmod p = g^{ab} \bmod p$$

Attacker challenge: discrete logarithm / computational DH hardness.

6.2 RSA signature workflow

Keygen:

- $n = pq$.
- $\phi(n) = (p-1)(q-1)$.
- Pick public e with $\gcd(e, \phi(n)) = 1$.
- Private d such that $de \equiv 1 \pmod{\phi(n)}$.

Sign message M :

$$S = M^d \bmod n$$

Verify:

$$M' = S^e \bmod n$$

Accept iff $M' = M$.

7) CAP Theorem (How to Answer)

During partition, cannot guarantee both full Consistency and full Availability. Practical framing for cloud: partition tolerance is mandatory, decide CP vs AP behavior by business risk.

Answer template:

1. State partition scenario.
 2. If correctness-critical $\rightarrow$ CP tendency (may block).
 3. If responsiveness-critical $\rightarrow$ AP tendency (allow staleness).
 4. Tie to real systems:
 - ReCAPTCHA-like large-scale collection path often AP-leaning.
 - Instant search suggestion path strongly AP-leaning.
-

8) PageRank

8.1 Core equation (no teleport)

$$r_i = \sum_{j \rightarrow i} \frac{r_j}{d_j}, \quad \sum_i r_i = 1$$

where d_j is out-degree of node j .

8.2 Damped PageRank

Teleport probability $(1 - d)$:

$$\mathbf{r} = dM\mathbf{r} + (1 - d)\frac{1}{n}\mathbf{1}$$

Common $d \approx 0.85 - 0.9$.

8.3 Fast approximation from old rank

If teleport changes slightly, one-step approximation:

$$\mathbf{r}' \approx d\mathbf{r} + (1 - d)\frac{1}{n}\mathbf{1}$$

Then comment: good approximation if damping change is small.

9) Consistent Hashing (Event Trace Method)

9.1 Core rule

Place server tokens and data hashes on ring $[0, M - 1]$. Each key goes to nearest clockwise server token.

9.2 Exam workflow

1. Compute all server token positions (including replicas/vnodes).
2. Sort ring clockwise.
3. For each data item, compute all replica hash positions.
4. Map each position clockwise to server.
5. Apply events in order (add server, fail server, add key).
6. Recompute only affected intervals.

9.3 Rebalance intuition

- Add server: mainly keys in predecessor arc move to new server.
- Remove server: only keys mapped to removed server move clockwise.
- Good property: localized remapping, not global remapping.

9.4 Load-balance risk formula

Probability n random points lie in some semicircle:

$$P = \frac{n}{2^{n-1}}$$

Meaning: uneven token clustering can create load skew; using many vnodes reduces this risk.

10) Last-Minute Memory Grid

Topic	Must write in exam
Clos	$r = N/n$, strict nonblocking $m \geq 2n - 1$, rearrangeable $m \geq n$
Fat-tree	hosts = $k^3/4$, core = $(k/2)^2$
EDF/SEDF	$U = \sum s_i/p_i$, schedulable if $U \leq 1$
Credit scheduler	share by weight ratio
AIMD	$STT = \frac{W}{2} RTT$, $p = \frac{8}{3W^2}$, $T \propto 1/\sqrt{p}$
RAID	RAID0 any fail, RAID5 2+ fail, RAID6 3+ fail
DH	$s = g^{ab} \bmod p$
RSA sign	$S = M^d \bmod n$, verify $S^e \bmod n$
CAP	Under partition choose CP vs AP by business priority
PageRank	sum incoming rank shares + normalization
Consistent hash	clockwise successor mapping + localized remap

11) Time Management for 2-Hour Paper

1. First 5 min: mark all calculation-heavy parts.
2. Do formula questions first (AIMD, EDF, RAID, DH/RSA, hash ring).
3. For theory questions, use compare tables and tradeoff language.
4. If stuck in long trace question, show method table first (often earns method marks).
5. Final 10 min: check arithmetic and ranking/order statements.